Test Summary Report
Job Application Tracker - Studi Kasus End-to-End

Tester	[Syaifa Turrohman]
Aplikasi yang Diuji	Job Application Tracker (project fullstack pribadi)
Tanggal Testing	24 September 2026
Lingkungan	Docker Compose (Backend, Frontend, PostgreSQL), localhost
Fitur yang Diuji	Status Pipeline, Interview Notes & Reminder

1. Ringkasan Eksekutif

Pengujian dilakukan terhadap dua fitur dengan kompleksitas tertinggi pada aplikasi Job Application Tracker: Status Pipeline (transisi status lamaran) dan Interview Notes & Reminder (relasi one-to-many dengan cascade delete). Proses pengujian mencakup requirement analysis, perancangan test case menggunakan test coverage matrix, eksekusi manual, serta verifikasi multi-layer melalui UI, Network tab, API, dan query SQL langsung ke database.

Metrik	Nilai
Total skenario dirancang (Status Pipeline)	16 (4 status × 4 status)
Skenario tereksekusi	5 (sampel representatif)
Skenario Pass	5
Skenario Fail	0
Test case fitur Notes & Reminder	2 (TC-001, TC-003)
Bug ditemukan	1 (reminder tanpa validasi tanggal)
Catatan improvement	1 (konsistensi response API pada parent yang terhapus)

2. Requirement Analysis

2.1 Status Pipeline

• Perpindahan status tidak terkunci urutan user dapat memilih status apa pun (Applied, Interview, Offer, Rejected) kapan saja melalui dropdown.
• Tidak ada status yang bersifat final; semua transisi, termasuk dari Rejected atau Offer kembali ke status lain, diizinkan.
• Tidak ada validasi tambahan (misal wajib isi catatan) saat perubahan status.

2.2 Interview Notes & Reminder

• Relasi Note-Application menggunakan onDelete: Cascade pada schema database notes akan ikut terhapus otomatis saat lamaran induk dihapus.
• Tidak ada batasan jumlah notes per lamaran.
• Field reminderDate bersifat optional di database dan tidak divalidasi di frontend maupun backend, meskipun tipe notes adalah "Reminder".

3. Test Design Status Pipeline Coverage Matrix

Seluruh 16 kombinasi transisi status dirancang untuk memastikan cakupan penuh terhadap kemungkinan perubahan status. Lima skenario representatif (mencakup transisi maju, mundur, antar status akhir, dan same-to-same) dipilih untuk dieksekusi secara langsung.

No	Status Awal	Status Tujuan	Ket.
1	Applied	Interview	Dieksekusi
2	Applied	Offer	Dirancang
3	Applied	Rejected	Dirancang
4	Applied	Applied	Dieksekusi
5	Interview	Applied	Dirancang
6	Interview	Offer	Dieksekusi
7	Interview	Rejected	Dirancang
8	Interview	Interview	Dirancang
9	Offer	Applied	Dirancang
10	Offer	Interview	Dirancang
11	Offer	Rejected	Dieksekusi
12	Offer	Offer	Dirancang
13	Rejected	Applied	Dirancang
14	Rejected	Interview	Dieksekusi
15	Rejected	Offer	Dirancang
16	Rejected	Rejected	Dirancang

4. Hasil Eksekusi — Status Pipeline

No	Transisi	Status Code	Dashboard Sebelum	Dashboard Sesudah	Hasil
1	Applied → Interview	200	Applied=4 Interview=2	Applied=3 Interview=3	Pass
2	Interview → Offer	200	Interview=3 Offer=0	Interview=2 Offer=1	Pass
3	Rejected → Interview	200	Rejected=2 Interview=2	Rejected=1 Interview=3	Pass
4	Offer → Rejected	200	Offer=1 Rejected=1	Offer=0 Rejected=2	Pass
5	Applied → Applied	(tidak ada request terkirim)	Applied=3	Applied=3	Pass

Kesimpulan: seluruh skenario yang dieksekusi berjalan sesuai ekspektasi. Dashboard statistik konsisten mengikuti perubahan status karena dihitung ulang langsung dari data asli (bukan counter manual), termasuk pada skenario same-to-same yang tidak mengirim request apa pun ke backend.

5. Test Case — Interview Notes & Reminder

TC-001: Reminder Tanpa Tanggal Jatuh Tempo

Test Scenario	Verifikasi validasi field reminderDate saat menambahkan reminder
Precondition	Aplikasi berjalan, minimal 1 data lamaran kerja tersedia
Test Steps	1. Buka salah satu card lamaran kerja 2. Expand panel "+ NOTES & REMINDERS" 3. Pindah ke tab "Reminders" 4. Isi field catatan ("Follow up soal hasil interview") 5. Pilih tipe "Reminder"

	6. Biarkan field tanggal kosong 7. Klik "Set reminder"
Test Data	content = "Follow up soal hasil interview", type = REMINDER, reminderDate = (kosong)
Expected Result	Sistem menolak submit, menampilkan pesan error ("Tanggal reminder wajib diisi"), data tidak tersimpan
Actual Result	POST /api/applications/7/notes mengembalikan 201 Created. Reminder tersimpan di tabel Note (id=5, type=REMINDER) dengan reminderDate=NULL (terverifikasi via query SQL dan Network tab). Tidak ada pesan error di UI.
Status	Fail (terverifikasi live)

TC-003: Cascade Delete pada Relasi Application-Note

Test Scenario	Verifikasi cascade delete pada relasi Application dan Note
Precondition	Lamaran id=8 memiliki 2 Note terkait: id=8 (INTERVIEW_NOTE) dan id=7 (REMINDER), dikonfirmasi via GET /api/applications/8/notes (2 item)
Test Steps	1. Catat notes lamaran id=8 (SQL & GET /api/applications/8/notes) 2. Hapus lamaran id=8 via UI, konfirmasi dialog 3. Catat status code request DELETE 4. Query SQL tabel Note & Application filter id=8 5. Kirim GET /api/applications/8/notes dan GET /api/applications/8
Test Data	applicationId = 8, dengan 2 Note terkait
Expected Result	DELETE berhasil (204). Kedua Note ikut terhapus dari database (0 rows). Lamaran tidak lagi ditemukan.
Actual Result	DELETE /api/applications/8 → 204 No Content. GET /api/applications/8/notes → [] (sebelumnya 2 item). GET /api/applications/8 → {"error": "Application tidak ditemukan"}. Query SQL tabel Note → 0 rows.
Status	Pass

6. Pengujian API Lamaran Kerja

Pengujian API dilakukan dua tahap: eksplorasi manual dengan Postman untuk memahami perilaku endpoint, lalu diotomasi dengan Playwright (*fixture request*) agar dapat dijalankan ulang. Test yang membuat data membersihkan datanya sendiri setelah selesai. Test dijalankan pada project Chromium.

6.1 Hasil Test API

ID	Method	Endpoint	Skenario	Hasil yang Diharapkan	Status
API-01	GET	/applications	Mengambil daftar lamaran	200, respons berupa array	Pass
API-02	POST	/applications	Membuat lamaran baru	Berhasil; status awal APPLIED	Pass
API-03	PUT	/applications/:id	Mengubah status ke INTERVIEW	Berhasil; terverifikasi ulang lewat GET	Pass

API-04	DELETE	/applications/:id	Menghapus lamaran	204; lamaran hilang dari daftar	Pass
API-05	GET	/applications/999999	Id angka yang tidak ada	404	Pass
API-06	POST	/applications	Body kosong	400	Pass
API-07	PUT	/applications/:id	Status tidak valid tidak mengubah data	Status tetap APPLIED	Pass
API-08	GET	/applications/abc	Id bukan angka	Error klien (4xx)	Fail (BUG-JAT-002)
API-09	PUT	/applications/:id	Status NGAWUR (di luar enum)	Error klien (4xx)	Fail (BUG-JAT-003)

Catatan: API-08 dan API-09 ditandai test.fail() di Playwright sebagai kegagalan yang diketahui, sehingga suite tetap hijau dan test akan melaporkan perubahan begitu bug diperbaiki.

6.2 Pola Penanganan Input Tidak Valid

Hasil gabungan dari seluruh pengujian menunjukkan penanganan input tidak valid pada backend tidak merata:

Endpoint	Input Tidak Valid	Hasil	Penilaian
POST /applications	Body kosong	400	Benar
GET /applications/:id	Id angka yang tidak ada	404	Benar
GET /applications/:id	Id bukan angka (abc)	500	Salah
PUT /applications/:id	Status di luar enum (NGAWUR)	500	Salah
POST /applications/:appId/notes	Reminder tanpa tanggal	201	Tidak divalidasi

Pada route GET /:id, kode memanggil Number() pada id lalu meneruskannya ke Prisma tanpa validasi, dan blok catch menjawab 500 tanpa mencatat error ke log. Pola serupa diduga berlaku pada route PUT, tetapi route tersebut belum diperiksa langsung.

7. Bug Report

BUG-JAT-001: Reminder Dapat Disimpan Tanpa Tanggal Jatuh Tempo

Severity	Major
Priority	High
Steps to Reproduce	1. Buka salah satu lamaran kerja 2. Tambahkan note baru dengan tipe "Reminder" 3. Isi konten catatan, biarkan field tanggal kosong 4. Submit
Expected Result	Backend menolak request (400) jika type = REMINDER namun reminderDate tidak dikirim
Actual Result	Request diterima (201 Created), reminder tersimpan dengan reminderDate = NULL
Dampak	Fungsi inti reminder (mengingatkan user pada waktu tertentu) gugur jika tanggal boleh kosong — reminder menjadi tidak berbeda secara fungsional dari catatan biasa (interview note). Berisiko menimbulkan bug lanjutan apabila ke depannya dibangun fitur seperti "reminder jatuh tempo hari ini"

	atau notifikasi otomatis, karena data reminderDate = NULL tidak dapat diproses oleh logic yang mengasumsikan tanggal selalu tersedia.
Rekomendasi	Tambahkan validasi di frontend (nonaktifkan tombol submit jika tipe Reminder namun tanggal kosong) dan di backend (POST /api/applications/:appId/notes mengembalikan 400 jika type = REMINDER tapi reminderDate tidak dikirim).

BUG-JAT-002: GET /api/applications/:id mengembalikan 500 untuk id yang bukan angka

Severity / Priority	Minor / Medium
Alasan	Fungsi normal tidak terganggu dan pengguna UI tidak terdampak, tetapi perbaikannya murah (beberapa baris validasi) dan log yang kosong menyulitkan debugging di seluruh endpoint.
Environment	Docker Compose (backend di localhost:5000), diuji lewat Postman dan Playwright
Steps to Reproduce	1. Pastikan aplikasi berjalan (docker compose up) 2. Kirim GET http://localhost:5000/api/applications/abc
Expected Result	400 Bad Request dengan pesan yang menyebut format id tidak valid
Actual Result	500 Internal Server Error, body {"error": "Gagal mengambil data application"}
Kontrol	GET /api/applications/999999 → 404 Not Found (id angka yang tidak ada ditangani dengan benar)
Analisis Penyebab	Dari membaca kode route GET /:id: Number('abc') menghasilkan NaN, diteruskan ke prisma.application.findUnique yang menolaknya. Error ditangkap blok catch dan dijawab 500 tanpa console.error, sehingga tidak ada jejak di log backend.
Bukti Tambahan	Test otomatis di api.spec.js (ditandai test.fail()); docker compose logs backend tidak memuat baris error untuk request ini.
Dampak	Status 500 menyesatkan untuk kesalahan format id dari klien; blok catch generik menyembunyikan detail error sehingga mempersulit debugging dan logging; tidak ada risiko kerusakan data (operasi baca), tetapi berpotensi memicu false alarm pada monitoring server.
Rekomendasi	1. Validasi id sebagai bilangan bulat sebelum query dan kembalikan 400 jika tidak valid 2. Catat error asli di blok catch (console.error)

BUG-JAT-003: PUT /api/applications/:id mengembalikan 500 untuk nilai status di luar enum

Severity / Priority	Minor / Medium
Alasan	Pengguna UI tidak terdampak karena dropdown hanya menawarkan nilai yang valid, tetapi respons 500 menyesatkan dan menyulitkan pemantauan, serta perbaikannya murah.
Environment	Docker Compose (backend di localhost:5000), diuji lewat Postman dan Playwright
Steps to Reproduce	1. POST http://localhost:5000/api/applications dengan body JSON {"company": "Postman Test", "position": "QA"} → 201, catat id dari response

	2. PUT http://localhost:5000/api/applications/{id} dengan body JSON {"status": "NGAWUR"}
Expected Result	400 Bad Request dengan pesan yang menyebut nilai status tidak valid
Actual Result	500 Internal Server Error, body {"error": "Gagal update application"}
Kontrol	PUT dengan {"status": "INTERVIEW"} → 200 OK
Bukti Tambahan	Test otomatis di api.spec.js (ditandai test.fail()); test terpisah memastikan status lamaran tetap APPLIED setelah request gagal (Pass).
Dampak	Klien atau frontend menerima 500 yang menyesatkan; blok catch diduga menangkap semua jenis error secara umum dan menyembunyikan detail error (route PUT belum diperiksa langsung); integritas data tetap aman karena data tidak berubah saat request gagal.
Rekomendasi	1. Validasi nilai status terhadap enum sebelum query dan kembalikan 400 2. Catat error asli ke log

Catatan Improvement: Konsistensi Response API

Setelah lamaran dihapus, GET /api/applications/:id mengembalikan error "tidak ditemukan", sedangkan GET /api/applications/:id/notes mengembalikan array kosong ([]). Dua endpoint untuk parent resource yang sama memberi sinyal berbeda, sehingga klien tidak dapat membedakan "lamaran ada tapi belum memiliki notes" dari "lamaran tidak ada". Disarankan agar endpoint notes juga mengembalikan 404 ketika parent application sudah tidak ada.

8. Kesimpulan & Rekomendasi

Fitur Status Pipeline berjalan baik pada seluruh skenario yang dieksekusi, termasuk edge case same-to-same yang ditangani tanpa request yang tidak perlu. Cascade delete pada relasi Application-Note juga bekerja sesuai desain. Pengujian UI dan API yang diotomasi dengan Playwright berjalan stabil dan membersihkan datanya sendiri.

Tiga bug ditemukan. Satu di antaranya signifikan (BUG-JAT-001, validasi reminder), sedangkan dua lainnya berdampak kecil tetapi berasal dari pola yang sama: input tidak valid sampai ke database lalu dijawab 500 oleh blok catch yang tidak mencatat error

Rekomendasi prioritas perbaikan:

1. BUG-JAT-001: validasi reminderDate (Major/High)
2. BUG-JAT-002 dan BUG-JAT-003: tambahkan validasi input dan console.error di blok catch (Minor/Medium); keduanya dapat diperbaiki bersamaan
3. Konsistensi response API pada endpoint notes setelah parent dihapus (improvement)